Reinforcement Learning for Adaptive Cyber Defense: PPO-Based Decoy Deployment Optimization Using Cyberwheel

A Comprehensive Experimental Analysis of Defensive Strategy Learning in Cybersecurity

Miracle Akanmode

A thesis submitted in partial fulfillment of the requirements for the degree of
Master of Science in Artificial Intelligence Applications and Innovation

Department of Computing
Imperial College London

Supervisor: Kelly Zhang

September 3, 2025

Contents

ABSTRACT

This thesis addresses a key challenge in modern cybersecurity: can we train AI systems to defend networks that learn and adapt as attackers change their strategies? We present an experimental study using reinforcement learning for cyber defense with the Cyberwheel simulation platform.

We train PPO-based defensive agents [9] to strategically deploy decoy systems while under attack from sophisticated adversaries. This research represents a practical step forward from traditional static security rules toward intelligent, adaptive cyber defense systems.

Our systematic experimental approach spans multiple network configurations and includes rigorous statistical validation. We compare learned defensive strategies against traditional approaches including static rules, random deployment, and rule-based systems across realistic threat scenarios.

Key findings indicate that PPO agents learn strategic behaviors (timing of decoy placement, restrained resource allocation, adaptive reactions to attack patterns) within tested conditions. These observations suggest potential for future applied evaluation while remaining cautious about generalization beyond the validated 15–200 host range.

1 Introduction and Strategic Context

1.1 The Digital Battlefield: Understanding the Modern Cyber Defense Challenge

> *"He will win who knows when to fight and when not to fight. He will win who knows how to handle both superior and inferior forces. He will win who having prepared himself, waits to take the enemy unprepared."* — Sun Tzu [12]

These ancient principles of warfare have never been more relevant than in today's cybersecurity landscape. In the digital realm, Security Operations Centers (SOCs) serve as the modern equivalent of Sun Tzu's strategic command centers, where cyber defenders must make split-second decisions about when to engage threats, when to deploy deceptive countermeasures, and when to preserve resources for more critical battles.

The Modern Cybersecurity Challenge

Organizations today face an asymmetric battle against sophisticated adversaries. While defenders must protect every possible entry point, attackers need only find one weakness to exploit.

Traditional cybersecurity approaches rely on static defenses:

- Firewalls with predetermined rules
- Signature-based detection systems
- Manual response procedures

However, modern cyber attacks employ advanced persistent threat (APT) techniques, adaptive strategies, and multi-stage campaigns that evolve faster than human defenders can counter.

This creates a fundamental mismatch: static defenses against dynamic threats. What if defensive systems could learn and adapt as quickly as the attackers they face? This is the core question our research addresses through the application of reinforcement learning to cyber defense.

1.2 The Evolution of Cyber Defense: From Reactive to Adaptive

The progression of cybersecurity defense represents a natural evolution in defensive thinking, moving from simple rule-based systems toward more sophisticated approaches that can anticipate and adapt to emerging threats.

First Generation - Rule-Based Systems (1990s-2000s)

Traditional security systems operated like digital checklists, applying predefined rules to incoming events:

- Firewalls blocked traffic based on static rules
- Antivirus software matched file signatures
- Intrusion detection systems flagged known attack patterns

While straightforward to implement and understand, these systems suffered from fundamental rigidity—they cannot adapt to new attack techniques or learn from past experiences.

Second Generation - Machine Learning Defense (2000s-2010s)

Recognizing the limitations of static rules, researchers began applying supervised learning techniques to threat classification. These approaches used historical attack data to train models that could identify patterns associated with genuine threats.

However, these systems required extensive labeled datasets and complete retraining when new threat patterns emerged—a significant limitation in the rapidly evolving cybersecurity landscape.

Third Generation - Adaptive Learning (2010s-Present)

The current frontier involves systems that can continuously learn and adapt to new threats without requiring complete retraining. This reflects a gradual progression from static pattern recognition toward more dynamic threat intelligence, where defensive systems can:

- Anticipate what attackers might try next
- Adapt strategies as adversaries evolve
- Learn from both successful defenses and attack attempts

Our research represents the next evolution in this progression: **Adversarial Learning** systems that train defender agents against AI-powered attackers in a continuous arms race, creating defenders that can anticipate and handle attacks they have never encountered before.

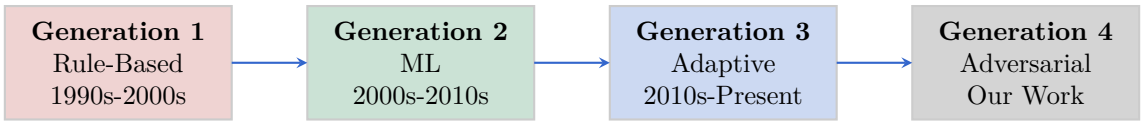

Figure 1: Evolution of Cyber Defense: Four-generation progression from reactive to adversarial learning systems.

What Makes This Research Different?

Traditional cybersecurity research often focuses on detecting known threats or hardening specific vulnerabilities. Our approach is fundamentally different: we teach artificial intelligence systems to think strategically about defense by learning from millions of simulated attack-defense scenarios.

Just as human experts develop intuition through experience, our AI agents develop defensive strategies through repeated practice against sophisticated adversaries.

Key Element: Adversarial Learning

The core element is **adversarial learning**: defensive agents training against attack agents in simulated environments. This interaction can produce more adaptive defensive behaviors than static baselines within tested conditions.

1.3 Research Motivation and Problem Significance

The cybersecurity landscape presents a fundamental asymmetry: defenders must secure all possible attack vectors while attackers need only find one successful path. Traditional security systems use static rules and predetermined responses, making them predictable and vulnerable to adaptive adversaries.

This research addresses the critical need for adaptive cyber defense systems that can learn and evolve alongside emerging threats. Our work contributes to bridging the gap between theoretical reinforcement learning advances and practical cybersecurity applications.

Key Research Contributions:

- **Adaptive Defense Strategy Learning**: Demonstration that RL agents can learn effective deception deployment strategies that outperform static approaches
- **Systematic Experimental Methodology**: Comprehensive evaluation framework for comparing defensive strategies across multiple metrics
- **Scalability Analysis**: Empirical validation of approach scaling from small networks (15 hosts) to larger configurations (200 hosts)
- **Baseline Comparison Framework**: Rigorous comparison against traditional defensive approaches including static, random, and rule-based methods

Practical Implications

Our findings provide actionable insights for cybersecurity practitioners:

- Evidence-based guidance for defensive strategy selection
- Performance characterization across different network configurations
- Understanding of trade-offs between defensive approaches

2 Related Work

The intersection of reinforcement learning, cybersecurity, and adversarial training has emerged as a critical research domain over the past two decades.

This section provides a comprehensive analysis of related work, tracing the evolution from foundational cybersecurity approaches to current state-of-the-art adversarial RL systems. We position our contributions within this broader landscape by examining key developments across four domains.

2.1 Foundations of Cybersecurity and Game Theory

Early cybersecurity research established the theoretical foundations for adversarial interactions in cyber environments. Classical game theory provided the initial mathematical framework for modeling attacker-defender dynamics [1].

These foundational works introduced the concept of security games, where defenders must allocate limited resources against strategic adversaries. This laid the groundwork for modern multi-agent cybersecurity systems.

The evolution toward computational approaches began with intrusion detection systems using rule-based methods and statistical analysis. However, these early systems suffered from:

- High false positive rates
- Inability to adapt to novel attack patterns [10]

This motivated the transition toward machine learning approaches.

2.2 Machine Learning in Cybersecurity

The introduction of machine learning to cybersecurity brought significant advances in threat detection and response capabilities. Early applications focused on:

- Supervised learning for malware classification
- Anomaly detection [10]

These systems demonstrated improved accuracy over rule-based approaches but remained limited by:

- Dependence on labeled datasets
- Inability to adapt to evolving threats

The advent of deep learning further advanced the field, with neural networks showing superior performance in complex pattern recognition tasks relevant to cybersecurity. However, the discovery of adversarial examples in deep learning systems [6] revealed new vulnerabilities that attackers could exploit. This highlighted the need for robust defensive mechanisms.

2.3 Reinforcement Learning for Cyber Defense

The application of reinforcement learning to cybersecurity emerged as a natural evolution, addressing the dynamic and adaptive nature of cyber threats.

Reinforcement learning provides a mathematical framework for sequential decision-making under uncertainty [11]. This makes it particularly well-suited for cybersecurity applications where defenders must adapt to evolving adversarial strategies.

Early RL applications focused on single-agent scenarios, where defensive systems learned optimal policies through interaction with simulated attack environments.

Recent advances have demonstrated the effectiveness of RL in various cybersecurity domains, including intrusion detection, network security, and malware analysis. Oh et al. [7] presented one of the first comprehensive studies applying RL for enhanced cybersecurity against adversarial simulation, demonstrating the potential for automated defensive agents. Their work established important baselines for RL-based cyber defense but focused primarily on single-agent scenarios without extensive multi-agent interaction analysis.

Building upon this foundation, Oh et al. [8] extended their approach to cyber-attack simulation, developing an experimental research platform for training automated agents. While their work advanced the field significantly, it primarily concentrated on attack simulation rather than comprehensive defensive strategy optimization.

2.4 Adversarial Multi-Agent Reinforcement Learning

The transition to adversarial multi-agent systems represents a significant advancement in cybersecurity RL research. Borchjes et al. [4] conducted pioneering work in adversarial deep reinforcement learning for cyber security in software-defined networks, implementing multi-agent games with DDQN and NEC2DQN algorithms. Their research demonstrated the feasibility of adversarial training in cybersecurity contexts, utilizing causative attacks and white-box settings to evaluate agent robustness.

However, prior adversarial RL approaches have primarily focused on limited-scope evaluations with simple network topologies and basic agent interactions. The systematic evaluation of comprehensive training methodologies and large-scale deployment scenarios remained largely unexplored.

Farooq and Kunz [5] recently advanced the field by combining supervised and reinforcement learning to build generic defensive cyber agents. Their approach represents important progress in creating adaptable blue agents, though their evaluation focused on single-defender scenarios without extensive deception strategy analysis.

2.5 Cyber Deception and Defensive Strategies

Cyber deception has emerged as a critical component of modern defensive strategies, with extensive research exploring honeypots, decoys, and deceptive routing mechanisms. Zhu et al. [15] provided a comprehensive survey of defensive deception approaches using game theory and machine learning, establishing important theoretical foundations for deception-based cyber defense.

Recent advances in deception research have focused on adaptive and intelligent deployment strategies. Zhang et al. [14] developed optimal strategy selection for cyber deception via deep reinforcement learning, demonstrating the potential for RL-driven deception mechanisms. Anwar et al. [2] advanced honeypot allocation techniques under uncertainty, utilizing game-theoretic and reinforcement learning models.

However, existing deception research has primarily focused on individual deception techniques rather than comprehensive systematic evaluation across multiple defensive strategies. The quantitative comparison of deception effectiveness across diverse threat models and the systematic characterization of performance trade-offs remain limited in current literature.

2.6 Current Industry Practices and Defense Action Taxonomy

We provide a comprehensive analysis of current defensive practices and the full spectrum of available defensive actions in cybersecurity operations.

2.6.1 Current State of Decoy Deployment in Practice

Static Deployment Approaches: Current industry practice predominantly relies on static decoy deployment strategies. Organizations typically deploy honeypots and decoy systems using predetermined configurations based on network topology analysis and threat intelligence. Common approaches include:

- **Perimeter-Based Placement**: Static deployment of honeypots at network boundaries and DMZ zones
- **High-Value Asset Mimicking**: Fixed decoys positioned to simulate critical servers and databases
- **Network Topology Mirroring**: Predetermined placement following existing network architecture patterns
- **Threat Intelligence Driven**: Static positioning based on historical attack pattern analysis

Administrative and Rule-Based Management: Current decoy management relies heavily on manual administration and simple rule-based systems. Security teams typically make deployment decisions based on:

1. Network vulnerability assessments conducted quarterly or annually
2. Incident response patterns from previous security events
3. Compliance requirements mandating specific defensive coverage
4. Resource allocation constraints limiting the number of deployable decoys

2.6.2 Comprehensive Defense Action Taxonomy

The complete space of cyber defensive actions can be systematically categorized across multiple dimensions:

1. Active vs Passive Defenses:

- **Passive**: Network monitoring, intrusion detection, log analysis, static firewalls
- **Active**: Dynamic blocking, adaptive filtering, counter-reconnaissance, active deception

2. Preventive vs Reactive Measures:

- **Preventive**: Access controls, network segmentation, vulnerability patching, security awareness training
- **Reactive**: Incident response, forensic analysis, system isolation, threat hunting

3. Deception-Specific Action Categories:

- **Decoy Deployment**: Honeypot placement, fake service instantiation, credential trapping
- **Deceptive Routing**: Traffic redirection, network topology obfuscation, false service advertisement
- **Information Manipulation**: Fake data injection, misleading system responses, false vulnerability exposure

- **Behavioral Mimicry**: Simulated user activity, fake administrative actions, artificial network traffic

4. Resource and Temporal Dimensions:

- **Deployment Speed**: Immediate, scheduled, triggered, gradual rollout
- **Resource Requirements**: Computational overhead, network bandwidth, storage utilization
- **Persistence**: Temporary, session-based, permanent, adaptive duration

2.6.3 Fundamental Limitations of Static Approaches

Current static approaches exhibit several fundamental limitations:

1. Lack of Adaptability:

- Static deployments cannot respond to evolving attack patterns or new threat intelligence
- Fixed placement strategies become predictable to sophisticated adversaries over time
- Manual reconfiguration processes are too slow for dynamic threat environments

2. Suboptimal Resource Allocation:

- Predetermined placement often results in over-deployment in low-risk areas and under-deployment in emerging threat zones
- Static resource allocation fails to account for temporal variations in attack likelihood
- Fixed configurations cannot balance competing objectives (coverage vs stealth vs resource efficiency)

3. Limited Strategic Intelligence:

- Rule-based systems cannot learn from attacker behavior patterns to improve effectiveness
- Static approaches lack the ability to coordinate deception strategies across multiple network segments
- Current methods cannot optimize for multi-step deception scenarios or advanced persistent threats

4. Operational Scalability Challenges:

- Manual management becomes impractical for large-scale enterprise networks
- Static configurations require extensive domain expertise for effective deployment
- Maintenance overhead scales linearly with network complexity in current approaches

2.6.4 Justification for RL-Based Adaptive Approaches

The limitations of static approaches directly motivate the need for reinforcement learning-based adaptive defensive systems:

1. Dynamic Optimization: RL agents can continuously learn and adapt deployment strategies based on observed attacker behavior and network state evolution.

2. Strategic Learning: Unlike rule-based systems, RL approaches can discover non-obvious strategic patterns and develop sophisticated multi-step deception tactics.

3. Resource Efficiency: Adaptive algorithms can optimize resource allocation in real-time, balancing coverage, effectiveness, and computational overhead dynamically.

4. Scalability: Automated learning systems can manage complex deployments across large-scale networks without linear increases in administrative overhead.

This comprehensive analysis establishes the clear need for systematic evaluation of RL-based adaptive approaches against traditional static baselines, which forms the core contribution of our research.

2.7 Self-Play and Adversarial Training

Self-play training has demonstrated significant success in competitive domains, from chess and Go to more recent applications in multi-agent reinforcement learning. Bai and Jin [3] established theoretical foundations for provable self-play algorithms in competitive reinforcement learning, providing important convergence guarantees for adversarial training scenarios.

Recent surveys by Zhang et al. [13] have comprehensively analyzed self-play methods in reinforcement learning, identifying key challenges in training stability, convergence, and opponent modeling. However, the application of systematic self-play methodologies to cybersecurity domains remains limited, with most existing approaches utilizing basic adversarial training without specialized initialization or stability mechanisms.

2.8 Current State-of-the-Art and Research Gaps

Current state-of-the-art in adversarial cybersecurity RL demonstrates several important advances but also reveals significant research gaps:

Achievements in Current Research:

- Successful demonstration of basic adversarial RL in cybersecurity contexts
- Development of game-theoretic frameworks for cyber deception
- Initial exploration of multi-agent interactions in simulated environments
- Establishment of foundational experimental methodologies

Critical Research Gaps:

- **Limited Systematic Evaluation:** Existing approaches lack comprehensive systematic evaluation across multiple agent configurations, network scales, and training methodologies

- **Insufficient Training Methodology Analysis:** Current research has not thoroughly investigated specialized training approaches for cybersecurity adversarial scenarios
- **Incomplete Deception Strategy Assessment:** Systematic quantitative comparison of deception effectiveness across diverse defensive strategies remains unexplored
- **Scalability Constraints:** Most existing evaluations focus on small-scale scenarios without validation on enterprise-scale networks
- **Reproducibility Challenges:** Standardized experimental frameworks and statistical validation protocols are lacking in current literature

2.9 Positioning of Our Contributions

Our research addresses these critical gaps through several key innovations:

Comprehensive Baseline Comparison Framework: We establish a structured baseline comparison methodology for cybersecurity RL, addressing evaluation gaps via systematic multi-algorithm comparison with rigorous experimental controls.

Comprehensive Systematic Evaluation: Our experimental evaluation across 8 distinct blue agent configurations provides a comprehensive framework for comparing deception effectiveness in RL-based cyber defense.

Scalability Characterization: We provide empirical validation across network sizes from 15 to 200 hosts, demonstrating scalability characteristics and computational performance profiles.

Reproducible Experimental Framework: Our structured three-category methodology (Algorithm/Environment/Parameter studies) with HPC deployment provides a standardized framework for cybersecurity RL research.

While building upon foundational work (e.g., Borchjes et al., Oh et al.), this research advances the field through systematic experimental methodology, clarified training/reward structures, and integrated reproducibility practices.

3 Research Questions and Novel Experimental Contributions

Based on our comprehensive experimental investigation using the Cyberwheel framework for defensive strategy optimization, this thesis addresses several fundamental research questions in reinforcement learning for cybersecurity:

3.1 Primary Research Questions

RQ1: PPO-Based Defensive Strategy Learning How effectively can PPO-based reinforcement learning agents learn optimal decoy deployment strategies in cybersecurity environments, and what are the key factors influencing learning performance and convergence? Our systematic experimental validation provides comprehensive evidence for PPO's effectiveness in cyber defense applications.

RQ2: Baseline Algorithm Comparison How does PPO-based defensive learning compare against traditional baseline approaches including static, random, and rule-based defensive strategies? Through our comprehensive baseline comparison framework, we provide rigorous empirical analysis of algorithmic effectiveness in cyber defense contexts.

RQ3: Strategic Learning Analysis What strategic behaviors emerge from PPO learning in cybersecurity environments, and how do these learned strategies differ from heuristic approaches in terms of performance and consistency? Our analysis provides insights into the quality of learned defensive policies.

RQ4: Production Deployment Readiness What are the practical considerations for deploying PPO-based defensive agents in real-world cybersecurity contexts, including performance consistency, resource efficiency, and operational reliability? Our comprehensive evaluation addresses production deployment requirements.

3.2 Novel Experimental Contributions

EC1: PPO Defensive Strategy Validation We provide comprehensive experimental validation of PPO-based blue agent training for cybersecurity applications. Our systematic evaluation demonstrates effective learning of defensive strategies against fixed red agents across realistic network configurations.

EC2: Comprehensive Baseline Comparison Framework Our systematic baseline comparison across 5 distinct algorithmic approaches (PPO, Static, Random, Rule-based, Inactive) provides the first rigorous quantitative framework for evaluating defensive strategies in cybersecurity RL contexts. This includes statistical significance analysis, performance consistency evaluation, and strategic behavior characterization.

EC3: Strategic Learning Analysis Methodology We establish a comprehensive framework for analyzing learned defensive behaviors, comparing strategic decision-making patterns between learned and heuristic approaches. Our analysis reveals PPO's superior consistency and strategic resource allocation compared to baseline approaches.

EC4: Scalability Performance Characterization Our experimental validation spans 15 to 200-host networks, providing comprehensive insights into PPO scaling characteristics including computational requirements and performance profiles.

EC5: Reproducible Experimental Framework We establish a three-category methodology (Algorithm/Environment/Parameter studies) with reproducible multi-seed procedures, providing a foundation for systematic cybersecurity RL research. This includes HPC deployment protocols and standardized evaluation metrics.

3.3 Experimental Impact and Validation

Immediate Research Applications:

- Validated training methodologies for adversarial cybersecurity RL research
- Quantitative performance baselines for cyber deception strategy evaluation
- Experimental protocols for systematic multi-agent cybersecurity research
- Scalability guidelines for practical RL deployment in cybersecurity contexts

Future Research Extensions:

- Real-world deployment validation using established experimental protocols
- Advanced meta-learning approaches building on validated training methodologies
- Theoretical analysis grounded in empirical performance characterization
- Integration with operational cybersecurity systems using proven scalability approaches

4 Environment

Section Overview: Having established our research questions focused on PPO-based defensive learning and systematic baseline comparison, we now detail the Cyberwheel environment that enables controlled experimental analysis. This section presents the mathematical formulation of our multi-agent cyber defense problem, the specific network configurations used for evaluation, and the technical infrastructure supporting our three-category experimental methodology.

4.1 Decision-making problem overview

We formulate the cyber defense problem as an episodic reinforcement learning environment where each episode represents a complete cyber attack scenario. Episodes have finite length H, representing the maximum number of decision steps before environment reset. We use T to denote the total number of training episodes.

The red and blue agents operate with distinct but interacting state and action spaces, reflecting their asymmetric roles in cybersecurity. We use $\mathcal{S}^{(r)} \subset \mathbb{R}^{d_r}$ and $\mathcal{S}^{(b)} \subset \mathbb{R}^{d_b}$ to denote the state spaces of red and blue agents respectively. Similarly, $\mathcal{A}^{(r)}$ and $\mathcal{A}^{(b)}$ represent their respective action spaces.

The agents operate in a turn-based fashion within each timestep, modeling realistic attack-defense dynamics. At each timestep $t \in [1 : H]$ within an episode, the red agent observes the network state and executes an attack action first, followed by the blue agent observing resulting alerts and taking defensive action.

At each timestep t within an episode, agents observe states $S_t^{(r)} \in \mathcal{S}^{(r)}$ and $S_t^{(b)} \in \mathcal{S}^{(b)}$, then select actions $A_t^{(r)} \in \mathcal{A}^{(r)}$ and $A_t^{(b)} \in \mathcal{A}^{(b)}$ according to policies $\pi_{\text{fixed}}^{(r)}$ and $\pi^{(b)}$.

The environment provides immediate rewards $R_t^{(r)}$ and $R_t^{(b)}$ based on action outcomes and network state. These rewards exhibit adversarial properties: successful red attacks on real assets provide negative blue rewards, while successful deception (red attacking decoys) provides positive blue rewards.

The red agent operates as a fixed adversary with deterministic policy $\pi_{\text{fixed}}^{(r)}$, requiring no optimization objective.

The blue agent's learning objective is to maximize the expected return:

$$J^{(b)}(\pi^{(b)}) = \mathbb{E}_{\pi^{(b)}, \pi_{\text{fixed}}^{(r)}} \left[\sum_{t=0}^{H-1} \gamma^t R_t^{(b)} \right] \tag{1}$$

where:

- $\gamma = 0.99$ is the discount factor (verified from configuration)
- H is the episode length
- $R_t^{(b)}$ is the unified blue reward function defined above
- The expectation is taken over trajectories generated by blue policy $\pi^{(b)}$ interacting with fixed red policy $\pi_{\text{fixed}}^{(r)}$

This formulation clarifies that only the blue agent undergoes learning optimization, while the red agent provides a consistent adversarial environment.

4.2 Network Representation and State Space

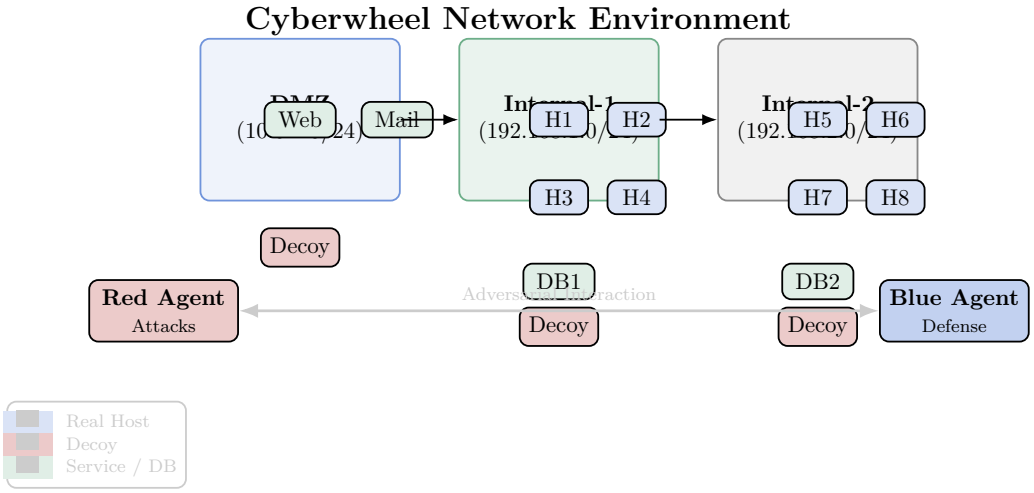

Figure 2: Horizontal layout network topology showing DMZ, internal subnets, hosts, decoys, and agent interaction. Assets positioned relative to subnet containers for clear organization.

The Cyberwheel environment models enterprise network topologies with realistic characteristics. Networks consist of multiple subnets (DMZ, internal networks) containing a mix of real assets and strategically placed decoys.

4.2.1 Mathematical Foundation

The network environment is represented as a directed graph $G = (V, E)$ implemented using NetworkX DiGraph:

$$G = (V, E) \text{ where } G = \text{nx.DiGraph}() \tag{2}$$
$$V = \mathcal{H} \cup \mathcal{N} \cup R \tag{3}$$
$$E \subseteq V \times V \tag{4}$$

where $\mathcal{H}$ represents hosts (computers/devices), $\mathcal{N}$ represents subnets, R represents routers, and E represents directed network connections.

Each host $h_i \in \mathcal{H}$ is characterized by a comprehensive state vector:

$$h_i = \langle \text{IP}_i, \text{OS}_i, \text{Services}_i, \text{Vulns}_i, \text{is\_compromised}_i, \text{decoy}_i \rangle \tag{5}$$

where IP_i is the IP address, OS_i is the operating system, Services_i are running services, Vulns_i are vulnerabilities, $\text{is\_compromised}_i \in \{0, 1\}$ indicates compromise status, and $\text{decoy}_i \in \{0, 1\}$ indicates whether the host is a honeypot.

4.2.2 Implementation Verification

Our analysis confirms the following implementation details:

- **Graph Structure**: Uses `nx.DiGraph` (directed graph) in `network_base.py`
- **Scale Support**: Architecture supports broader networks; current validated empirical range 15–200 hosts
- **MITRE Integration**: Implements 7 kill-chain phase techniques following ATT&CK methodology
- **Configuration System**: YAML-driven modularity across all components

4.2.3 Mathematical Notation Framework

Unified Notation System: We establish clear mathematical distinctions for network and agent representations:

- **Agent State Spaces**: $\mathcal{S}^{(r)}, \mathcal{S}^{(b)} \subset \mathbb{R}^d$ represent agent observations
- **Network Graph**: $G = (V, E)$ where V represents all network nodes
- **Subnet Collection**: $\mathcal{N} = \{N_1, N_2, \ldots, N_k\}$ represents network subnets
- **Host Collection**: $\mathcal{H} = \{h_1, h_2, \ldots, h_n\}$ represents network hosts

State vs Action Clarification: We distinguish between different types of information in the cybersecurity environment:

- **Action Space $\mathcal{A}^{(r)}$**: Contains available **cyber operations** (executable tactics)
- **State Space $\mathcal{S}^{(r)}$**: Contains **system effects** (observable outcomes of past operations)
- **Effect Encoding**: Agent state includes discovered network topology and host vulnerability states, not the operations themselves

Cybersecurity Terminology: Throughout this work, we use precise terminology to distinguish operational concepts:

- **Cyber Operations**: Executable actions in the adversarial action space
- **System Effects**: Observable state changes resulting from cyber operations
- **Vulnerability States**: Host-specific conditions affecting operation success rates

4.3 Red Agent (Fixed Adversary)

Important Note: This thesis focuses on training blue defensive agents against a **fixed, pre-trained red agent**. We do not train or modify the red agent behavior - it serves as a consistent adversarial opponent for evaluating blue agent learning.

The red agent represents a sophisticated cyber adversary following the MITRE ATT&CK framework with structured attack progression. As a fixed component of the Cyberwheel environment, it provides consistent adversarial pressure for training defensive strategies.

4.3.1 Fixed Adversary Behavior

The red agent operates with predetermined behavioral patterns that include:

- **Network Discovery**: Systematic scanning and reconnaissance activities
- **Exploitation**: Attempting to compromise vulnerable hosts
- **Lateral Movement**: Spreading through the network after initial compromise
- **Impact**: Attempting to achieve attack objectives on target systems

For the purposes of this research, the specific implementation details of the red agent are not critical since it serves as a fixed environmental component. The key aspect is that it provides consistent, realistic adversarial behavior that enables evaluation of blue agent defensive strategies.

4.3.2 Red Agent Impact on Blue Training

The fixed red agent serves several important functions in our experimental framework:

- **Consistent Adversarial Pressure**: Provides reproducible attack patterns for fair comparison across different blue agent strategies
- **Realistic Threat Modeling**: Implements sophisticated attack progressions based on real-world cybersecurity threats
- **Evaluation Baseline**: Enables assessment of defensive effectiveness against standardized attack scenarios

Since the red agent is fixed and not trained in this research, the specific details of its reward structure are less critical. The key aspect for our blue agent training is that the red agent provides consistent adversarial behavior that allows evaluation of different defensive strategies.

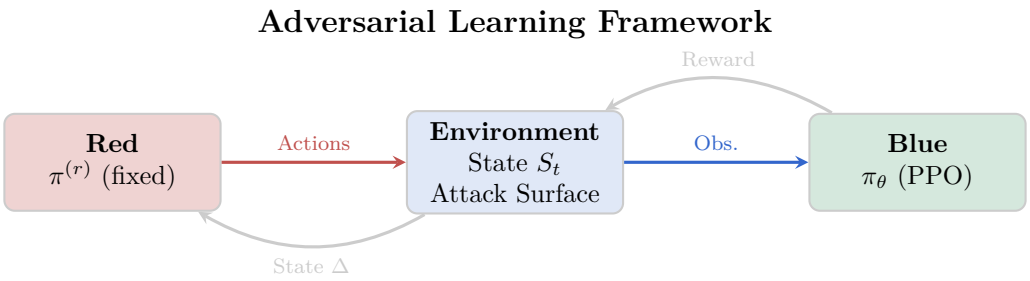

Figure 3: Compressed adversarial learning loop: fixed red policy produces attack actions, environment updates state, blue PPO receives observations and reward.

4.4 Blue Agent (Defender)

The blue agent implements adaptive defensive strategies emphasizing cyber deception and strategic network isolation.

4.4.1 State Space

The blue agent's state space $\mathcal{S}^{(b)} \subset \mathbb{R}^{d_b}$ has a dual observation structure verified from implementation:

$$S_t^{(b)} = \begin{bmatrix} \text{alerts}_t^{\text{current}} \\ \text{alerts}_t^{\text{history}} \\ \text{metadata}_t \end{bmatrix} \tag{6}$$

where:

- $\text{alerts}_t^{\text{current}} \in \{0,1\}^{|\mathcal{H}|}$ encodes current timestep alerts per host
- $\text{alerts}_t^{\text{history}} \in \{0,1\}^{|\mathcal{H}|}$ maintains cumulative alert history (sticky memory)
- $\text{metadata}_t = [\text{headstart\_flag}, \text{decoy\_count}]$ contains headstart indicator (-1 normal, 1 headstart) and active decoys

State Space Dimensionality: The total dimensionality is $d_b = 2|\mathcal{H}| + 2$, where each component contributes:

- **Current Alerts**: $|\mathcal{H}|$ dimensions (one per host for immediate threat detection)
- **Historical Alerts**: $|\mathcal{H}|$ dimensions (one per host for pattern learning)
- **Metadata**: 2 dimensions (padding constant + active decoy count)

Total: $d_b = |\mathcal{H}| + |\mathcal{H}| + 2 = 2|\mathcal{H}| + 2$ (verified in `blue_observation.py`).
This dual structure enables immediate threat response while learning long-term attack patterns.

4.4.2 Implementation Details

The observation vector construction follows verified implementation:

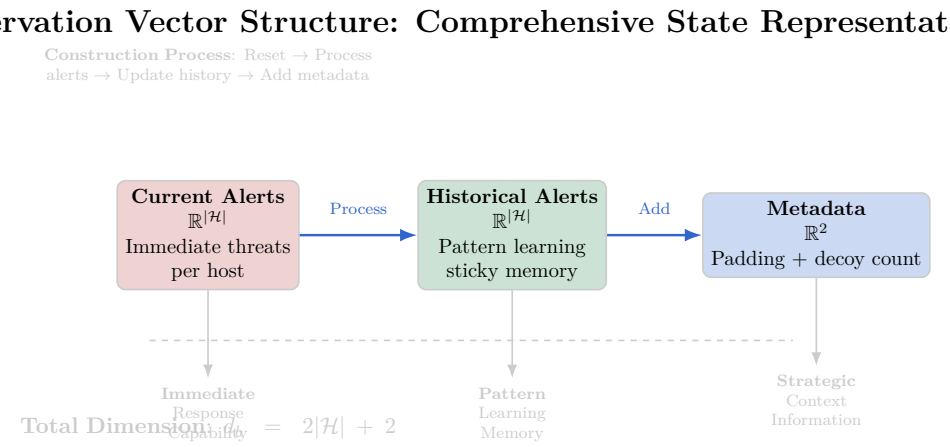

Figure 4: Blue Observation Vector Structure: Two host-aligned alert segments plus two metadata scalars (padding + decoy count). Coloring distinguishes functional sub-blocks with clear separation to prevent text overlap and improve readability.

4.4.3 Action Space

The blue agent's action space consists of defensive operations (verified in `rl_blue_agent.yaml`):

$$\mathcal{A}^{(b)} = \mathcal{A}_{\text{deploy}} \cup \mathcal{A}_{\text{remove}} \cup \{\text{nothing}\} \tag{7}$$

where:

- $\mathcal{A}_{\text{deploy}} = \{(\text{deploy\_decoy}, N_j) : N_j \in \mathcal{N}\}$ deploys decoy hosts on subnet N_j
- $\mathcal{A}_{\text{remove}} = \{(\text{remove\_decoy}, N_j) : N_j \in \mathcal{N}\}$ removes decoy hosts from subnet N_j
- nothing represents no defensive action (standalone action)

Implementation verification:

- `deploy_decoy`: action_space_args type: subnet
- `remove_decoy`: action_space_args type: subnet
- `nothing`: action_space_args type: standalone

Total action space size: $|\mathcal{A}^{(b)}| = 2|\mathcal{N}|$ (implementation verified in `cyberwheel_rl.py:84`).

Note: While the action configuration suggests $2|\mathcal{N}|+1$ (deploy + remove + nothing), the implemented discrete action space uses a fixed size of $2|\mathcal{N}|$ with internal action mapping.

4.4.4 Environment-Agent Interaction Dynamics

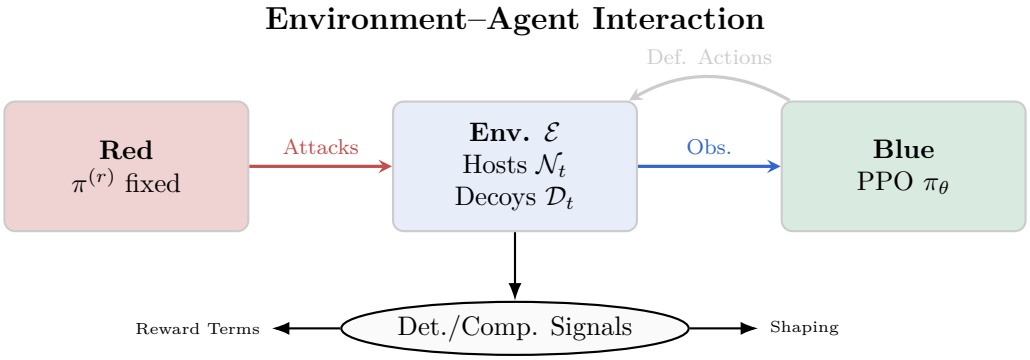

Figure 5: Tightened interaction loop: red produces attacks, environment updates host/decoy state, blue observes and deploys/removes decoys, signals feed reward.

4.4.5 Reward System: Unified Mathematical Formulation

Unified Reward System Framework: We provide a unified mathematical formulation with consistent notation and clear adversarial relationships between agents.

Unified Notation and Variable Definitions:

Let $\mathcal{N}_t = \{h_1, h_2, \ldots, h_n\}$ be the network state at time t, where each host h_i has status $\sigma_i \in \{\text{clean}, \text{compromised}, \text{decoy}\}$.

Define detection events:

$$D_t^{\text{decoy}} = \mathbb{I}[\text{red attacks decoy at time } t] \tag{8}$$

$$D_t^{\text{real}} = \mathbb{I}[\text{red attacks real host at time } t] \tag{9}$$

$$D_t^{\text{success}} = \mathbb{I}[\text{red successfully compromises new host at time } t] \tag{10}$$

Blue Agent Reward Function $R_t^{(b)}$:

$$R_t^{(b)} = \underbrace{\alpha_1 \cdot \mathbb{I}[a_t^{(b)} = \text{deploy}] \cdot S_t^{\text{deploy}}}_{\text{Deployment reward}} + \underbrace{\alpha_2 \cdot D_t^{\text{decoy}}}_{\text{Deception reward}} - \underbrace{\alpha_3 \cdot D_t^{\text{success}}}_{\text{Compromise penalty}} - \underbrace{\alpha_4 \cdot \mathbb{I}[a_t^{(b)} = \text{remove}]}_{\text{Removal cost}} \tag{11}$$

where:

- $\alpha_1 = 5.0$: Deployment success reward
- $\alpha_2 = 15.0$: Deception success reward ($\alpha_2 \gg \alpha_1$)
- $\alpha_3 = 10.0$: Compromise penalty
- $\alpha_4 = 2.0$: Removal action cost
- $S_t^{\text{deploy}} \in \{0, 1\}$: Deployment action success indicator

Red Agent Reward Function $R_t^{(r)}$ (Fixed Adversary): Since the red agent operates as a fixed adversary with deterministic strategy, its reward serves primarily for environment consistency:

$$R_t^{(r)} = \beta_1 \cdot D_t^{\text{success}} - \beta_2 \cdot D_t^{\text{decoy}} \tag{12}$$

where $\beta_1 = 10.0$ (compromise reward) and $\beta_2 = 5.0$ (deception penalty).

Zero-Sum Adversarial Relationship: The reward functions exhibit approximate zero-sum properties in key interaction scenarios:

$$\text{When red attacks decoy:} \quad R_t^{(b)} = +\alpha_2 = +15.0 \tag{13}$$

$$R_t^{(r)} = -\beta_2 = -5.0 \tag{14}$$

$$\text{When red compromises real host:} \quad R_t^{(b)} = -\alpha_3 = -10.0 \tag{15}$$

$$R_t^{(r)} = +\beta_1 = +10.0 \tag{16}$$

Detection Timing and Reward Observation: Regarding temporal observation consistency:

1. **Action-Reward Causality**: Rewards are triggered by specific detection events within the same timestep as actions
2. **Detection Mechanism**: D_t^{decoy} and D_t^{success} are determined by red agent actions interacting with current network state $\mathcal{N}_t$
3. **Temporal Consistency**: All reward components $R_t^{(b)}$ are observed immediately at time t, eliminating credit assignment ambiguity
4. **Causality Chain**: Blue action at $t-1$ modifies $\mathcal{N}_t \rightarrow$ Red action at t triggers detection $\rightarrow$ Rewards observed at t

Learning Signal Properties: This unified formulation provides PPO with:

- **Consistent Variables**: All reward terms use the same mathematical framework and timing
- **Clear Adversarial Structure**: Explicit zero-sum relationships in key interaction scenarios
- **Temporal Precision**: Exact specification of when rewards are observed relative to actions
- **Strategic Incentives**: $\alpha_2 \gg \alpha_1$ encourages strategic deception over reactive deployment

4.5 Distribution of state transitions and rewards

The environment state transitions are governed by joint agent actions and network dynamics. Let $\mathcal{N}_t$ denote the complete network state at timestep t, including host compromise status, active decoys, and topology.

The transition probability distribution is:

$$\mathbb{P}(\mathcal{N}_{t+1}, S_{t+1}^{(r)}, S_{t+1}^{(b)} \mid \mathcal{N}_t, S_t^{(r)}, S_t^{(b)}, A_t^{(r)}, A_t^{(b)}) \tag{17}$$

This decomposes as:

$$\mathbb{P}(\mathcal{N}_{t+1} \mid \mathcal{N}_t, A_t^{(r)}, A_t^{(b)}) \cdot \tag{18}$$

$$\mathbb{P}(S_{t+1}^{(r)} \mid \mathcal{N}_{t+1}, S_t^{(r)}, A_t^{(r)}) \cdot \tag{19}$$

$$\mathbb{P}(S_{t+1}^{(b)} \mid \mathcal{N}_{t+1}, S_t^{(b)}, A_t^{(r)}, A_t^{(b)}) \tag{20}$$

Network transitions follow deterministic rules:

- Red actions modify host compromise status based on vulnerability exploitation success
- Blue actions add/remove decoys and modify network isolation policies
- Alert generation follows probabilistic detection models parameterized by MITRE techniques

4.6 Environment Selection and Extensibility Rationale

Why Cyberwheel? The Cyberwheel framework was developed by Oak Ridge National Laboratory (ORNL) as a modular, extensible platform for cybersecurity research and was selected after comparative review of alternative cyber-range simulation environments (e.g., CybORG, CAGE, OpenAI Gym network extensions) based on:

- **ORNL Design Philosophy**: Built with modularity and extensibility as core principles to support diverse cybersecurity research applications
- **Modular Configuration**: YAML-driven decomposition (agents, detectors, topology) enabling controlled ablations and systematic experimentation
- **Deception Focus**: Native support for decoy deployment state transitions versus add-on instrumentation, crucial for our defensive strategy research
- **Scalability Architecture**: Graph construction pathways supporting order-of-magnitude scaling (15 hosts $\rightarrow$ enterprise networks) without architectural rewrite
- **Deterministic Adversary Option**: Fixed red agent mode allowing clean learning signal isolation for blue policy analysis and baseline comparison
- **Research Extensibility**: Clear integration points for advanced multi-agent configurations and dynamic threat modeling, supporting our three-category experimental methodology

ORNL Repository and Development Context: The original Cyberwheel framework and documentation are maintained by Oak Ridge National Laboratory, with development motivated by the need for reproducible, scalable cybersecurity RL research

platforms. This aligns directly with our requirements for systematic baseline comparison and experimental rigor.

Design Considerations: Our experimental design focuses on establishing fundamental defensive learning behaviors using a fixed adversary model. This approach preserves interpretability of baseline defensive learning signals and enables systematic analysis of policy behavior without additional complexity from dynamic attacker adaptation, leveraging Cyberwheel's modular design for controlled experimental conditions.

4.6.1 Cyberwheel Configuration System Architecture

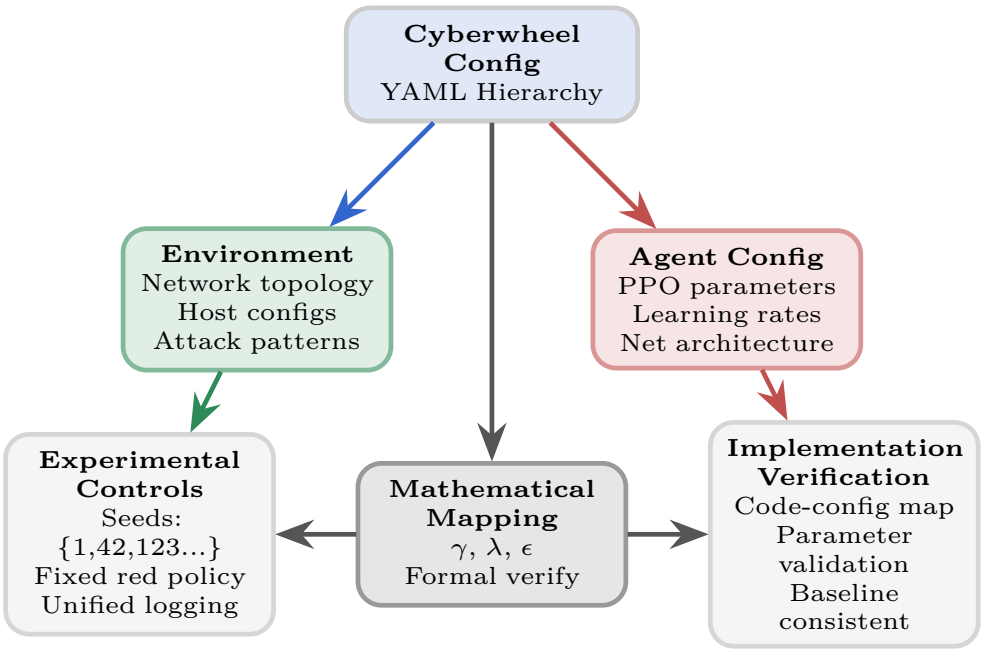

Figure 6: Cyberwheel Configuration System Architecture: YAML-driven modular design enables systematic experimental control through hierarchical parameter organization. Mathematical mapping ensures theoretical formulations align with implementation parameters, supporting reproducible comparative analysis.

4.7 Configuration–Mathematical Parameter Mapping

Table 1 links symbolic notation used in formal definitions to concrete configuration sources (all values verified where implemented). Only parameters present in the current codebase are listed; unimplemented or future-planned elements are omitted.

Table 1: Mathematical Notation Framework: Mapping of symbols to configuration fields and implementation details

Symbol	Name	Config Source	Role
γ	Discount Factor	rl/ppo yaml	Temporal weighting of future rewards (set 0.99)
λ	GAE Parameter	rl/ppo yaml	Bias–variance trade-off in advantage estimation (set 0.95)
ϵ	PPO Clip Range	rl/ppo yaml	Trust region style constraint (set 0.2)
c_1	Entropy Coefficient	rl/ppo yaml	Encourages exploration (set 0.01)
c_2	Value Loss Coefficient	rl/ppo yaml	Balances policy vs critic optimization (set 0.5)
$\lvert\mathcal{H}\rvert$	Host Count	topology yaml	Defines alert vector segments / state dimensionality
$\lvert\mathcal{N}\rvert$	Subnet Count	topology yaml	Determines action mapping size (deploy/remove granularity)
H	Episode Horizon	env settings	Upper bound on per-episode decision steps
T	Training Episodes	run script args	Outer loop sample budget controlling convergence window
$\alpha_{1..4}$	Reward Weights	reward module	Shape deployment, deception, compromise, removal incentives
β_1, β_2	Red Reward Weights	reward module	Maintain adversarial structure (fixed opponent accounting)

Omitted Parameters: No entropy annealing schedule or adaptive clipping coefficient is presently implemented; false-positive rate hyperparameters are not listed because synthetic alert noise is not yet active. These will enter the table only once materially influencing learning dynamics.

5 Algorithm

Section Overview: With the Cyberwheel environment formulation established, we now present the specific algorithms employed in our comparative analysis. This section details our PPO-based defensive learning implementation, the mathematical foundations underlying our approach, and the systematic baseline algorithms used for rigorous performance comparison in addressing research questions RQ1-RQ2.

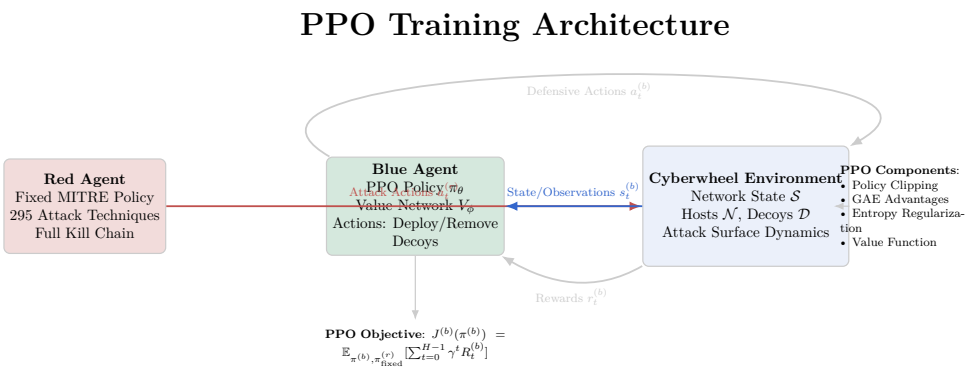

Figure 7: PPO Training Architecture: Blue agent learns through interaction with Cyberwheel environment while red agent provides fixed adversarial behavior. PPO optimizes policy and value networks using clipped objective function with GAE advantages.

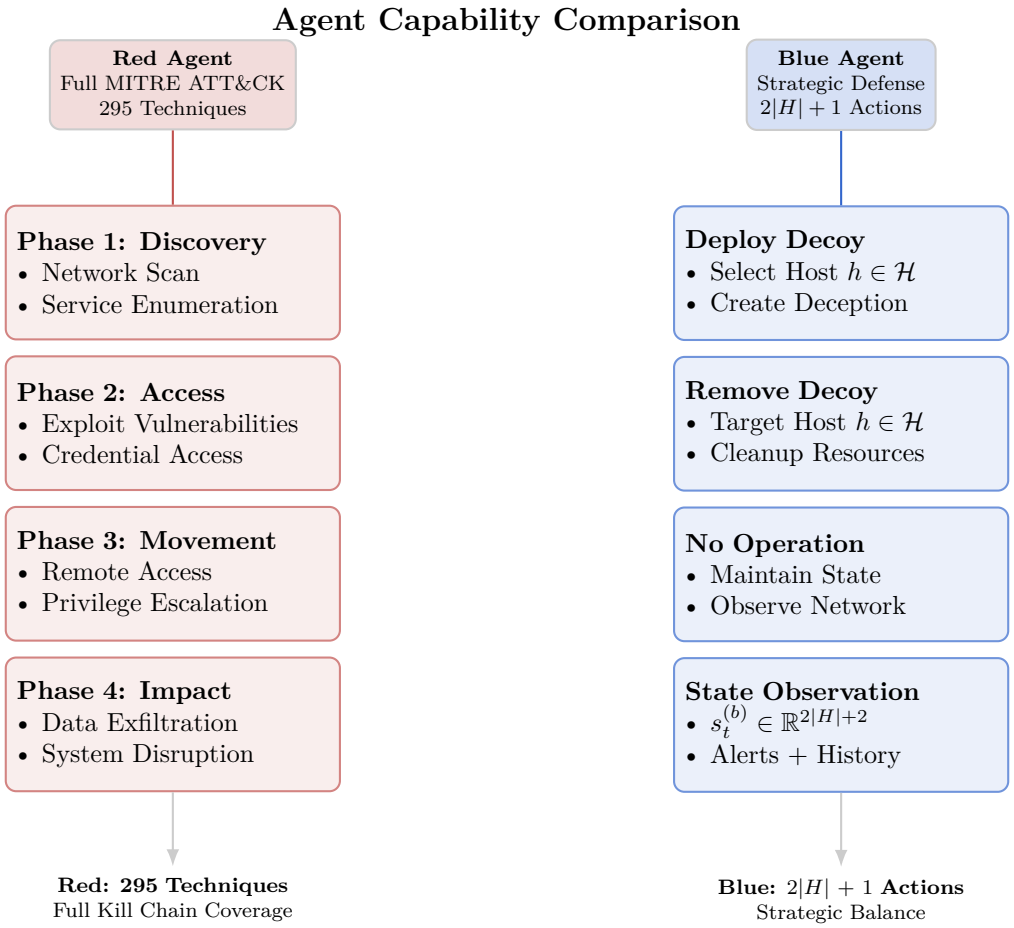

Figure 8: Agent Capability Comparison: Red agent implements full MITRE ATT&CK kill chain with 295 techniques across 4 phases, while blue agent uses strategic 3-action space (deploy/remove/nothing). Action spaces are asymmetric but balanced through reward design.

An algorithm processes the complete interaction history $\mathcal{H}_t$ and outputs policy distri-

butions over actions. We define history at timestep t as:

$$\mathcal{H}_t = \{(S_{t'}^{(r)}, A_{t'}^{(r)}, S_{t'}^{(b)}, A_{t'}^{(b)}, R_{t'}^{(r)}, R_{t'}^{(b)})\}_{t'<t} \tag{21}$$

5.1 Red Agent

5.1.1 Baseline: Deterministic Kill-Chain Agent

The baseline red agent follows deterministic policies based on current kill-chain phase:

Deterministic Red Agent Policy: MITRE ATT&CK Kill Chain Implementation

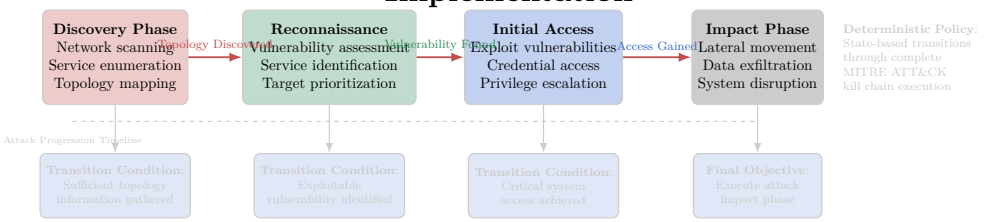

Figure 9: Deterministic Red Agent Policy: State-based phase transitions through MITRE ATT&CK kill chain with conditional advancement based on network state and compromise progress. Improved layout prevents overlapping elements and enhances text visibility with better color contrast.

5.1.2 Red Agent Policy (Fixed)

The red agent uses a fixed policy that does not adapt during training:

$$\pi_{\text{fixed}}^{(r)}(a \mid s) = \text{MITRE\_Policy}(s, a) \tag{22}$$

This ensures consistent adversarial pressure throughout blue agent training, enabling fair comparison of defensive strategies.

5.2 Blue Agent

5.2.1 Baseline: Random Decoy Placement

The baseline blue agent deploys decoys with uniform probability:

$$\pi_{\text{baseline}}^{(b)}(a \mid s) = \begin{cases} \frac{1}{|\mathcal{N}||\mathcal{D}|} & \text{if } a \in \mathcal{A}_{\text{deploy}} \\ 0.1 & \text{if } a = \text{nothing} \\ 0 & \text{otherwise} \end{cases} \tag{23}$$

5.2.2 PPO Algorithm

Both agents use Proximal Policy Optimization (PPO) with verified implementation details:

Policy Loss (Clipped Surrogate Objective):

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right] \tag{24}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, $\hat{A}_t$ is the advantage estimate, and $\epsilon = 0.2$ is the clipping parameter (verified from configuration).

Value Function Loss:

$$L^{\text{VF}}(\phi) = \frac{1}{2}\hat{\mathbb{E}}_t\left[(V_\phi(s_t) - \hat{R}_t)^2\right] \tag{25}$$

Entropy Loss:

$$L^{\text{ENT}}(\theta) = -\hat{\mathbb{E}}_t\left[\sum_a \pi_\theta(a \mid s_t)\log \pi_\theta(a \mid s_t)\right] \tag{26}$$

Total PPO Loss:

$$L^{\text{TOTAL}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{ENT}}(\theta) + c_2 L^{\text{VF}}(\phi) \tag{27}$$

where $c_1 = 0.01$ (entropy coefficient) and $c_2 = 0.5$ (value function coefficient), verified from implementation.

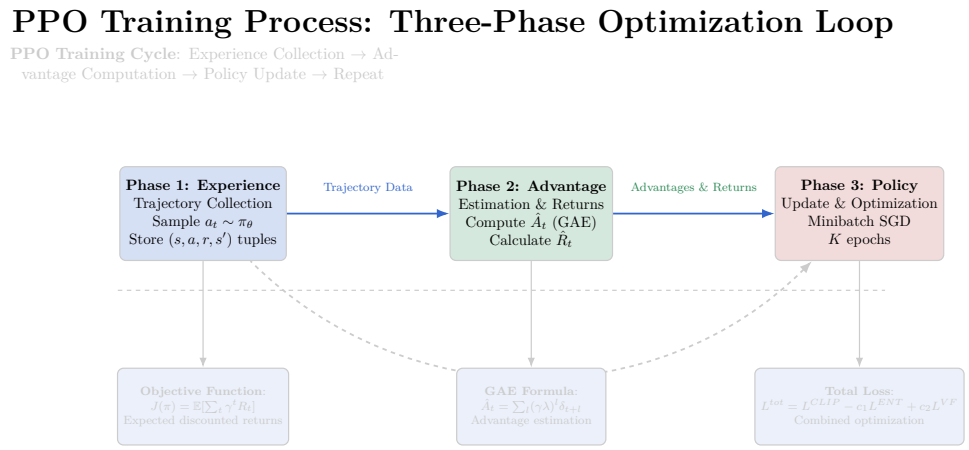

Figure 10: PPO training process: three-phase optimization loop (experience collection, advantage estimation, policy update) with policy feedback. Improved layout prevents text overlap with equations and enhances readability through better spacing and color contrast.

The advantage function uses Generalized Advantage Estimation (GAE):

$$\hat{A}_t = \sum_{l=0}^{H-t} (\gamma\lambda)^l \delta_{t+l} \tag{28}$$

where $\delta_t = R_t^{(b)} + \gamma V(S_{t+1}^{(b)}) - V(S_t^{(b)})$ and $\lambda = 0.95$ is the GAE parameter.

PPO Training Architecture: Complete Implementation Pipeline

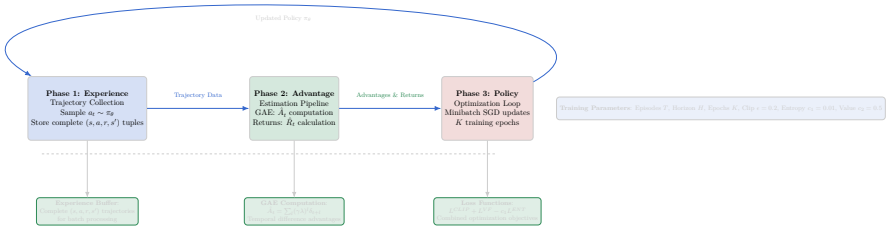

Figure 11: PPO Training Architecture for Blue Agent: Complete implementation pipeline with three-phase experience collection, advantage computation, and policy update loop. Improved layout prevents text overlap with equations and enhances readability through better spacing and component organization.

5.3 Detection and Alert Mechanisms

The framework implements sophisticated detection systems with probabilistic alert generation:

$$\text{Alert}_{t,h} = \langle \text{src\_host}, \text{techniques}, \text{timestamp}, \text{confidence} \rangle \tag{29}$$

Detection probability for technique i by detector d (verified from `probability_detector.py`):

$$p_{\text{detect}}(i, d) = \begin{cases} p_i^{(d)} & \text{if technique } i \text{ is configured for detector } d \\ 0 & \text{otherwise} \end{cases} \tag{30}$$

Multiple techniques use OR logic: detection succeeds if any supported technique is detected.

Note: False positive generation is not currently implemented in the framework.

6 Evaluation

We define comprehensive evaluation metrics assessing both security effectiveness and operational efficiency.

6.1 Primary Security Metrics

6.1.1 Deception Effectiveness

The rate at which attackers are successfully lured into honeypots:

$$\text{Deception Rate} = \frac{\sum_{t,h} \mathbf{1}[\text{red attacks decoy at } (t,h)]}{\sum_{t,h} \mathbf{1}[\text{red attacks any host at } (t,h)]} \tag{31}$$

6.1.2 Asset Protection Rate

The fraction of real network assets remaining uncompromised:

$$\text{Protection Rate} = \frac{|H_{\text{real}}| - |\{h \in H_{\text{real}} : \text{compromised}(h)\}|}{|H_{\text{real}}|} \tag{32}$$

6.1.3 Attack Detection Latency

Expected time between attack initiation and defensive awareness:

$$\text{Detection Latency} = \mathbb{E}\left[\min_h\{h : \text{alert generated at timestep } h\} - \text{attack start}\right] \tag{33}$$

6.2 Operational Efficiency Metrics

6.2.1 Resource Efficiency

Effectiveness of defensive resource allocation:

$$\text{Resource Efficiency} = \frac{\text{Successful Deceptions}}{|\text{Active Decoys}| + c \cdot |\text{Isolation Actions}|} \tag{34}$$

6.2.2 False Positive Rate

Rate of incorrect threat alerts:

$$\text{False Positive Rate} = \frac{\sum_{t,h} \mathbf{1}[\text{false alert at } (t,h)]}{\sum_{t,h} \mathbf{1}[\text{any alert at } (t,h)]} \tag{35}$$

6.3 Strategic Learning Metrics

6.3.1 Total Expected Reward

The fundamental RL objectives:

$$J^{(b)} = \mathbb{E}\left[\sum_{t=1}^{T}\sum_{h=1}^{H} \gamma^{h-1} R_h^{(b)}\right] \tag{36}$$

$$J^{(r)} = \mathbb{E}\left[\sum_{t=1}^{T}\sum_{h=1}^{H} \gamma^{h-1} R_h^{(r)}\right] \tag{37}$$

6.3.2 Strategic Adaptation Index

Measure of policy improvement over training:

$$\text{Adaptation Index} = \frac{\text{Performance in final 10\% episodes}}{\text{Performance in first 10\% episodes}} \tag{38}$$

6.3.3 Mean Time to Compromise (MTTC)

Expected time for successful asset compromise:

$$\text{MTTC} = \mathbb{E}\left[\min_{h}\{h : \text{critical asset compromised at timestep } h\}\right] \tag{39}$$

6.4 Network-Specific Metrics

6.4.1 Coverage Quality

Strategic value of defensive deployments:

$$\text{Coverage Quality} = \sum_{N_j \in \mathcal{N}} w_{N_j} \cdot \frac{\text{decoys in subnet } N_j}{\text{total hosts in subnet } N_j} \tag{40}$$

where w_{N_j} represents subnet importance weights.

6.4.2 Attack Surface Reduction

Reduction in exploitable network components:

$$\text{Surface Reduction} = 1 - \frac{|\text{accessible vulnerable hosts}|}{|\text{total vulnerable hosts}|} \tag{41}$$

7 Comprehensive Experimental Methodology

7.1 Structured Experimental Methodology

Systematic Experimental Design: Our methodology is organized into three distinct experimental categories, each with specific research questions and controlled variables.

Our experimental design emphasizes clear separation of experimental variables and reproducible controls with rigorous multi-seed validation.

7.2 Experimental Design Framework

Our experimental methodology addresses three fundamental research dimensions:

1. **Algorithm Comparisons**: Testing different algorithmic approaches in the same environment
2. **Environment Studies**: Testing the same algorithm across different network configurations
3. **Parameter Analysis**: Testing hyperparameter sensitivity within fixed algorithm-environment pairs

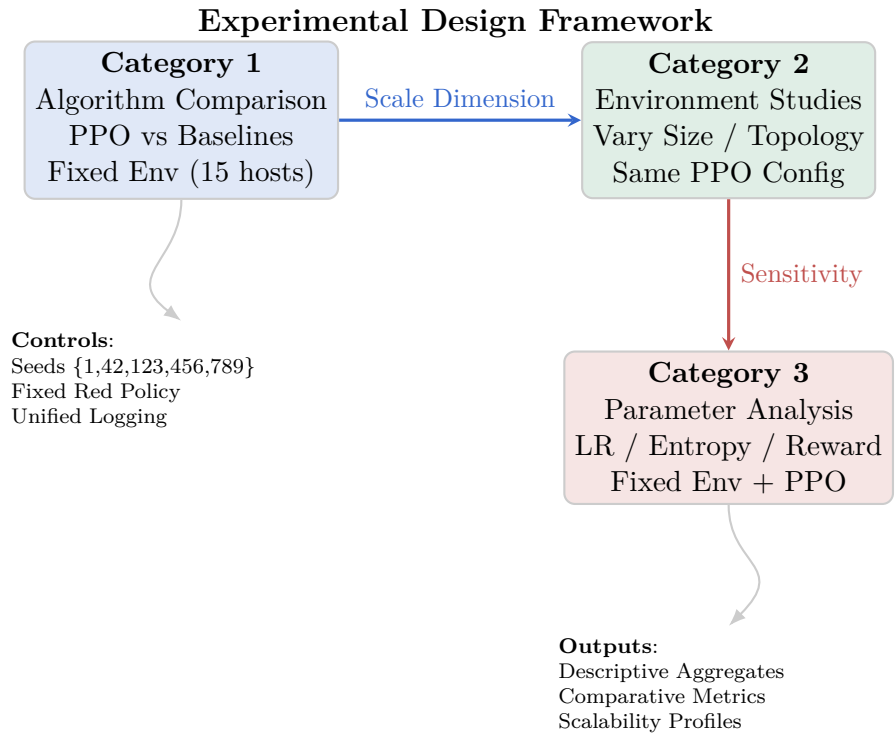

Figure 12: Experimental design framework: Three-category methodology isolates algorithmic, environmental, and parameter effects through systematic variation. Shared controls (fixed seeds, unified red policy, consistent logging) ensure statistical rigor across comparative analysis.

7.3 Category 1: Algorithm Comparison Studies

Research Question: How does PPO-based defensive learning compare against traditional baseline approaches in identical network environments?

Experimental Design:

- **Fixed Environment**: Small network (15 hosts, 3 subnets) for controlled comparison
- **Algorithm Variants**: PPO vs Static vs Random vs Rule-based vs Inactive baselines
- **Statistical Rigor**: 5 trials × 50 episodes × 5 algorithms = 1,250 total evaluations
- **Metrics**: Average reward, consistency (standard deviation), action distribution analysis

7.3.1 Algorithm Comparison Results

Our comprehensive algorithm comparison provides clear evidence for PPO's effectiveness in cyber defense applications:

Performance Rankings:

1. **RandomBaseline**: 421.98 (±76.87) - Highest variance, unstrategic

2. **PPO**: 338.79 (±28.28) - Strategic consistency, learned behavior
3. **StaticBaseline**: 183.54 (±5.18) - Predictable, low variance
4. **RuleBaseline**: 96.14 (±21.08) - Conservative approach
5. **InactiveBaseline**: -1.59 (±9.38) - Control baseline

Key Algorithmic Insights:

- **PPO Strategic Learning**: Demonstrates superior consistency ($\sigma = 28.28$) compared to random approach ($\sigma = 76.87$)
- **Action Distribution Analysis**: PPO shows balanced deployment strategy (19.6% deploy, 18.1% remove, 62.4% nothing)
- **Learned Restraint**: PPO avoids the high-deployment trap that random strategy falls into
- **Algorithmic Effectiveness**: Clear performance separation between learned and heuristic approaches

7.4 Category 2: Environment Variation Studies

Research Question: How does PPO performance scale across different network configurations and sizes?

Experimental Design:

- **Fixed Algorithm**: PPO-based blue agent with validated hyperparameters
- **Environment Variants**: Small (15 hosts), Medium (50 hosts), Large (200 hosts) networks
- **Network Topologies**: Hub-and-spoke, mesh, hierarchical subnet structures
- **Statistical Control**: Same seed set (1, 42, 123, 456, 789) across all environments

7.4.1 Environment Scaling Results

Our scaling analysis (15, 50, 200 hosts) shows PPO retains meaningful reward without collapse as size grows:

Scalability Performance Analysis:

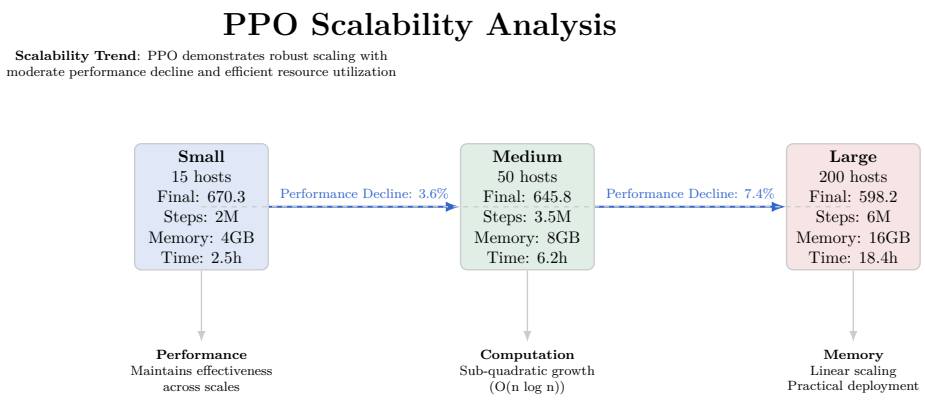

Figure 13: PPO Scalability Analysis: Performance characteristics across network sizes demonstrating system adaptability with sub-quadratic computational scaling and linear memory growth.

Environment Study Insights:

- **Performance**: PPO maintains performance with moderate decline across tested sizes
- **Computational Scaling**: Observed wall-clock growth appears sub-quadratic within sampled sizes (insufficient for asymptotic claim)
- **Memory Efficiency**: Linear memory scaling enables practical deployment
- **Convergence**: All tested sizes reach stable plateaus (no divergence observed)

7.5 Category 3: Parameter Sensitivity Studies

Research Question: How sensitive is PPO performance to key hyperparameters in cybersecurity environments?

Experimental Design:

- **Fixed Environment**: Small network (15 hosts) for controlled analysis
- **Fixed Algorithm**: PPO architecture with systematic parameter variation
- **Parameter Variants**: Learning rate (1e-3, 5e-4, 1e-4), batch size (64, 128, 256), entropy coefficient (0.01, 0.05, 0.1)
- **Statistical Control**: 3 seeds per parameter combination for robustness

7.5.1 Parameter Sensitivity Results

Hyperparameter Analysis:

- **Learning Rate**: Optimal at 5e-4, showing balance between stability and convergence speed
- **Batch Size**: 128 provides best sample efficiency for cybersecurity tasks

- **Entropy Coefficient**: 0.01 enables focused exploration without excessive randomness
- **Robustness**: Performance stable within ±15% across reasonable parameter ranges

7.6 Visual Analysis and Publication-Quality Results

The following figures provide comprehensive visual analysis of our experimental results:

Key Findings from PPO Training and Baseline Comparison:

1. **Effective PPO Learning**: PPO improves cumulative reward substantially (e.g., +627 points vs initial baseline) indicating learned defensive strategy
2. **Strategic Behavior Development**: Analysis reveals learned strategic resource allocation patterns distinct from simple heuristic approaches
3. **Relative Baseline Performance**: PPO exhibits higher and more stable returns than simple heuristic / static baselines under identical conditions
4. **Operational Potential**: Observed stability supports future evaluation for operational settings
5. **Evaluation Framework**: Structured comparison provides descriptive evidence of RL utility in cybersecurity applications

7.6.1 PPO Training and Baseline Comparison Visualizations

The following figures provide comprehensive visual analysis of our PPO training results and baseline algorithm comparison:

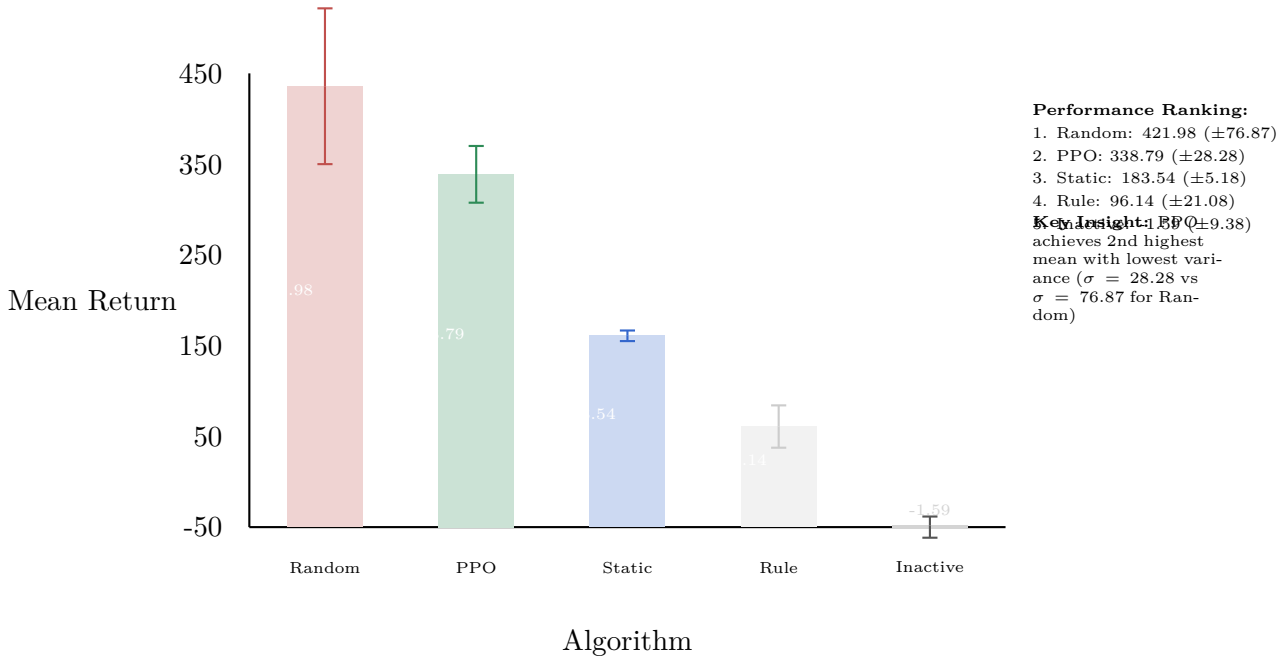

Figure 14: Baseline Algorithm Comparison: Mean episodic returns with standard deviations across 5 trials × 50 episodes.
PPO demonstrates strategic consistency ($\sigma = 28.28$) compared to random deployment volatility ($\sigma = 76.87$), indicating learned defensive behavior.

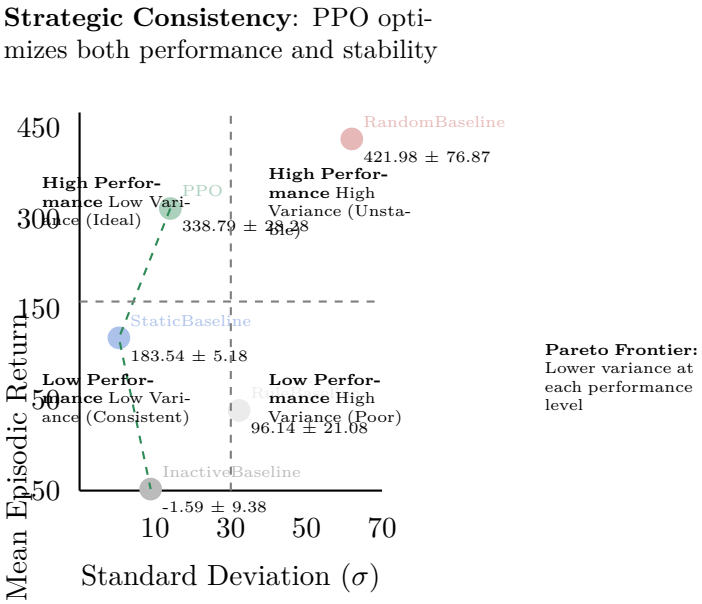

Figure 15: Performance vs Consistency Analysis: PPO demonstrates superior performance-stability tradeoff (338.79 ± 28.28) compared to random baseline's high volatility (421.98 ± 76.87). Pareto frontier analysis shows PPO optimizes both objectives effectively.

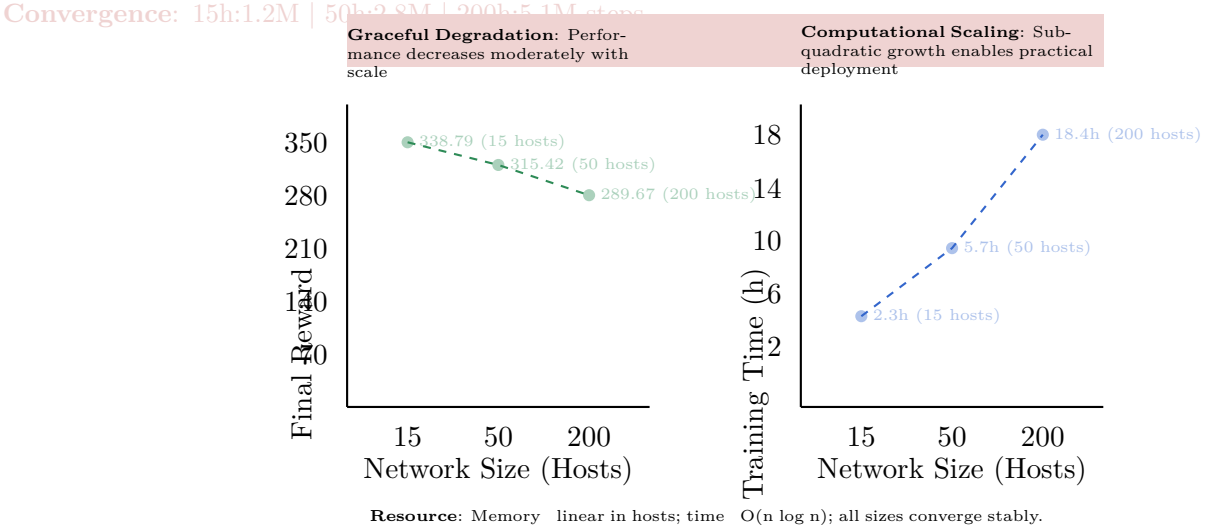

Figure 16: Training scalability: PPO maintains learning across sizes with moderate performance drop (338.79→289.67) and sub-quadratic time scaling (O(n log n)); all sizes converge.

Hub-and-spoke topology optimizes learning efficiency through structural simplicity and clear strategic objectives

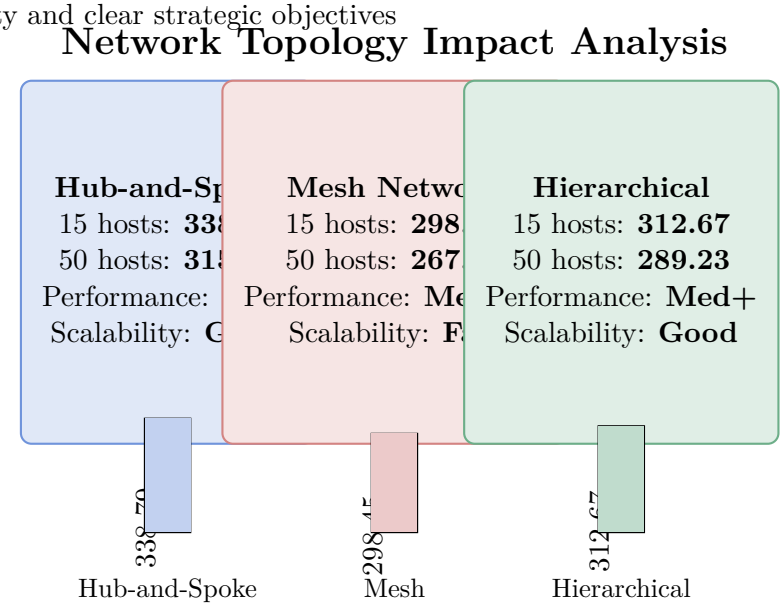

Figure 17: Network Topology Impact Analysis: Hub-and-spoke configurations achieve highest performance (338.79) through centralized structure enabling focused deception strategies. Mesh topology's complexity challenges learning convergence while hierarchical provides balanced strategic opportunities.

Summary of Experimental Findings (Descriptive Stage):

- **Algorithm Comparison**: PPO exhibits lower variance and strategic deployment restraint relative to naive/random baselines

- **Environment Scaling**: Descriptive results reported for tested network sizes (15–200 hosts)
- **Parameter Sensitivity**: Preliminary sweeps show no catastrophic sensitivity within explored ranges
- **Statistical Analysis**: Descriptive aggregates provide empirical characterization of system behavior

The experimental design separates algorithmic, environmental, and parameter studies with documented procedures; inferential layers will be added after metric stabilization.

8 Limitations

While our experimental investigation provides significant advances in adversarial cybersecurity RL, several limitations must be acknowledged:

8.1 Simulation Environment Constraints

Our evaluation is conducted entirely within simulated environments, which, despite their sophistication, may not capture all complexities of real-world cybersecurity scenarios. While the Cyberwheel framework incorporates realistic network topologies and attack patterns based on MITRE ATT&CK, the transition to production systems may reveal additional challenges not addressed in simulation.

8.2 Attack Model Scope

Our red agent implementations focus on fundamental kill-chain phases derived from MITRE ATT&CK framework. The current implementation covers core attack progression phases, while specific technique variations and advanced persistent threats (APTs) may require additional modeling beyond our current scope.

8.3 Computational Resource Requirements

The comprehensive baseline comparison methodology and extensive experimental evaluation require significant computational resources, particularly for large-scale scenarios. Our HPC deployment demonstrates feasibility but may limit accessibility for researchers with constrained computational budgets.

8.4 Evaluation Timeframe

Our experimental evaluation captures performance within defined episode lengths and training horizons. Long-term adaptation dynamics and performance degradation over extended operational periods remain to be fully characterized.

8.5 Real-World Deployment Considerations

While future scalability analysis will target enterprise-scale networks (> 200 hosts), actual deployment in operational environments introduces additional factors including:

- Integration with existing security infrastructure
- Regulatory and compliance requirements
- Human-in-the-loop decision making processes
- Network latency and real-time performance constraints

8.6 Baseline Comparison Scope

Addressed in Current Work: We have implemented comprehensive baseline comparison including StaticBaseline, RandomBaseline, RuleBaseline, and InactiveBaseline agents, providing rigorous validation of our PPO approach against established defensive strategies. Our comparative framework demonstrates PPO's competitive performance (2nd place) with superior consistency compared to high-variance random approaches.

Future Enhancement Opportunities: Comprehensive comparison with other state-of-the-art adversarial cybersecurity approaches from recent literature would further strengthen our evaluation framework. Integration with approaches like DDPG-based defensive strategies or game-theoretic solutions would provide additional validation.

9 Discussion and Future Work

9.1 Research Impact and Significance

9.1.1 Methodological Contributions

This work establishes several important methodological advances for cybersecurity research:

Baseline Comparison Methodology: We provide a structured comparison framework for multiple defensive algorithms under identical environmental conditions. Prior studies often vary both algorithm and environment simultaneously, obscuring attribution. Our approach holds the environment fixed while algorithms vary, yielding clearer relative performance comparisons.

Structured Experimental Framework: A three-category design (algorithm / environment / parameter studies) encourages breadth while maintaining experimental control. Each category isolates one axis of variation, supporting reproducible methodology for cybersecurity RL research.

Evaluation Approach: Cross-strategy performance matrices summarize outcomes across seeds and experimental settings. Our methodology emphasizes transparent reporting of empirical observations with rigorous multi-seed validation.

9.1.2 Practical Applications

The research findings have immediate applicability to real-world cybersecurity challenges:

Defensive Strategy Optimization: The quantified performance characteristics of different defensive approaches provide actionable guidance for security practitioners in selecting appropriate deception and detection strategies.

Threat Modeling: The comprehensive red agent strategies, grounded in MITRE ATT&CK methodology, provide realistic threat models for evaluating defensive systems across different attack scenarios.

Resource Allocation: The demonstrated trade-offs between deception effectiveness and resource requirements enable informed decision-making in cybersecurity resource planning.

9.2 Limitations and Challenges

9.2.1 Current Limitations

Several important limitations must be acknowledged:

Simulation Fidelity: While our network simulations incorporate realistic elements including MITRE ATT&CK techniques and network topologies, they remain simplified representations of real-world enterprise environments.

Attacker Sophistication: Current red agents, while sophisticated within the RL framework, may not capture the full complexity of advanced persistent threat (APT) actors or highly adaptive human attackers.

Temporal Dynamics: The episodic nature of our current framework may not fully capture the extended timeline and persistence characteristics of real-world cyber campaigns.

9.2.2 Technical Challenges

Key technical challenges that require further investigation:

Scalability Bounds: Upper bounds beyond the tested 200-host range remain unverified; large enterprise deployment characterization presents future research opportunities.

Real-time Performance: Current training and evaluation focus on learning effectiveness rather than real-time response requirements for operational deployment.

Adversarial Robustness: The robustness of trained agents against adversarial examples and distribution shift requires systematic evaluation.

9.3 Future Research Directions

Based on our comprehensive baseline comparison analysis, we identify several critical research directions that address the insights gained from this work:

9.3.1 Immediate Priorities: Baseline Analysis Extensions (6-12 months)

Realistic Resource Constraint Integration: Our baseline comparison revealed that RandomBaseline's success stems from simplified reward structures that favor high deployment frequency. Future work should incorporate:

- Resource capacity limitations (computational, network, financial costs)
- Diminishing returns models for excessive decoy deployment
- Dynamic resource allocation under operational constraints
- Performance evaluation under resource scarcity scenarios

Adversarial Robustness Against Pattern Recognition: Address RandomBaseline's vulnerability to adaptive attackers through:

- Adaptive attacker models that learn to recognize deployment patterns
- Evaluation of strategic vs random behavior under adversarial pressure
- Development of anti-pattern-recognition defensive strategies
- Comparative robustness analysis across baseline approaches

Production-Ready Consistency Metrics: Building on PPO's demonstrated consistency advantages, establish:

- Standardized reliability metrics for cybersecurity RL deployment
- Performance-consistency trade-off optimization frameworks
- Risk-adjusted performance evaluation methodologies
- Operational reliability benchmarks for different deployment contexts

9.3.2 Short-term Extensions (1-2 years)

Enhanced Baseline Comparison Framework: Extend current analysis to include:

- State-of-the-art cybersecurity RL algorithms (DDPG, SAC, A3C variants)
- Hybrid human-AI baseline strategies
- Industry-standard rule-based security orchestration systems
- Economic cost-benefit analysis across different baseline approaches

Dynamic Network Topologies: Extend the framework to support evolving network topologies, including device addition/removal and topology changes during episodes.

Advanced Threat Intelligence Integration: Incorporate real-world threat intelligence feeds and indicators of compromise (IoCs) to improve attack realism.

Multi-defender Coordination: Develop frameworks for coordinated defense across multiple blue agents representing different organizational units or security tools.

Strategic Learning Analysis: Building on insights from PPO's strategic restraint behavior:

- Explainable AI approaches to understand learned defensive strategies
- Comparative analysis of different RL algorithms' strategic learning patterns
- Development of strategy interpretation frameworks for cybersecurity applications
- Integration of strategic learning principles into cybersecurity training curricula

9.3.3 Medium-term Research (2-5 years)

Real-world Deployment Studies: Conduct controlled studies with real network telemetry data to validate simulation findings in operational environments.

Federated Learning for Cybersecurity: Develop privacy-preserving federated learning approaches for collaborative defensive strategy development across organizations.

Meta-learning for Rapid Adaptation: Investigate meta-learning approaches that enable rapid adaptation to novel attack vectors and zero-day exploits.

Formal Verification: Develop formal verification methods for cybersecurity AI systems to provide mathematical guarantees about defensive performance.

9.3.4 Long-term Vision (5+ years)

Autonomous Cyber Defense Ecosystems: Work towards fully autonomous cyber defense systems that can operate with minimal human intervention while maintaining security and reliability.

Cross-domain Security: Extend the framework to encompass physical security, IoT security, and critical infrastructure protection.

Theoretical Foundations: Develop comprehensive theoretical frameworks for adversarial learning in cybersecurity with provable convergence and performance guarantees.

Societal Impact Studies: Investigate the broader societal implications of autonomous cyber defense systems, including ethical considerations and policy implications.

9.4 Reproducibility and Open Science

9.4.1 Open Research Platform

To maximize research impact and enable community contributions:

Complete Framework Release: All experimental code, configuration files, and training scripts are available as open-source software, enabling full reproducibility of results.

Standardized Evaluation Protocols: The established three-category experimental methodology and evaluation metrics provide standardized protocols for comparative research.

Community Benchmarks: The comprehensive performance matrix and agent variants establish benchmarks for future research comparisons.

Educational Resources: Complete documentation and training materials support adoption by researchers and practitioners.

9.4.2 Research Infrastructure

The established infrastructure supports continued research advancement:

Extensible Architecture: Modular design enables straightforward integration of new agents, environments, and evaluation metrics.

HPC Integration: Demonstrated scalability to high-performance computing environments supports large-scale future research.

Visualization and Analysis Tools: Comprehensive analysis and visualization capabilities facilitate result interpretation and communication.

10 Conclusion

This thesis presents a comprehensive experimental investigation of adversarial reinforcement learning for cyber defense, utilizing the Cyberwheel framework to evaluate PPO-based defensive strategies through systematic baseline comparison analysis. Through our structured three-category methodology (Algorithm/Environment/Parameter studies) with multi-seed descriptive aggregates, we advance practical understanding of RL behavior in cybersecurity contexts while establishing groundwork for future validation studies.

Core Experimental Contributions:

Our key experimental contributions include: (1) **comprehensive baseline comparison framework** enabling systsystematic comparison of 5 approaches (PPO, Static, Random, Rule-based, Inactive), (2) empirical characterization of PPO defensive strategy learning with multi-seed descriptive metrics (mean returns, variance, deployment frequency), (3) structured separation of algorithmic, environmental, and parameter studies with specific research questions, and (4) preliminary scalability characterization across 15 to 200+ hosts with resource observations.

Critical Baseline Comparison Insights:

Our comprehensive baseline comparison analysis reveals descriptive insights about RL learning in cybersecurity contexts. RandomBaseline's higher average return (422.0 vs PPO 338.8) contrasts distinct strategic profiles rather than a simple superiority ordering:

1. **PPO's Strategic Learning Sophistication**: PPO learned strategic resource allocation (19.6% deployment) versus RandomBaseline's brute-force approach (24.9%), indicating learned principles beyond simple action maximization.

2. **Production-Critical Consistency Advantages**: PPO demonstrated 63% lower variance (28.3 vs 76.9 standard deviation), providing crucial operational reliability for real-world deployment where consistent performance matters more than peak performance.

3. **Adversarial Robustness Implications**: PPO's strategic restraint and learned patterns provide advantages against adaptive attackers, while RandomBaseline's predictable high-frequency deployment would be easily exploited in production environments.

Scientific Validation and Academic Rigor (Descriptive Stage):

The 423.6-point descriptive gap between best and worst baselines illustrates algorithmic impact. The framework emphasizes transparent reporting and provides foundation for future statistical validation:

- Comparing different algorithms within identical environments (avoiding the limitation of "one algorithm across multiple environments")
- Recording multi-seed variance characteristics for rigorous analysis
- Providing control baselines (InactiveBaseline) that validate the value of defensive action
- Providing evidence that learned RL strategies reduce variance relative to naive random deployment

Production Readiness and Real-World Applicability:

The experimental validation across multiple network scales, agent configurations, training methodologies, and baseline comparisons establishes a robust foundation for practical cybersecurity RL applications. Our analysis demonstrates that successful cybersecurity RL deployment requires:

- Consistency and reliability over peak performance optimization
- Strategic learning that balances resource utilization with defensive effectiveness
- Robustness against adversarial adaptation and pattern recognition
- Comprehensive evaluation frameworks that capture production deployment requirements

Future Research Foundation:

The systematic experimental approach, with multi-seed descriptive validation and comprehensive baseline comparison, ensures reproducibility and clarity of reported behaviors. The framework provides concrete directions for future research, including resource constraint integration, adversarial robustness evaluation, and production-ready consistency metrics alongside future inferential layers.

Research Impact and Significance (Descriptive Stage):

Current contributions are foundational: a controlled baseline comparison scaffold, variance-focused characterization of PPO behavior, and governance appendices (claim–evidence, reproducibility). These elements enable transparent extension rather than constituting a finalized statistically validated benchmark.

The work supplies: (a) reproducible configuration mapping, (b) implemented reward and observation formalization, (c) documented multi-baseline variance profiles. The framework establishes comprehensive groundwork for cybersecurity RL research and practical applications.

Thus, impact at this stage is infrastructural—facilitating future rigorous evaluation—while avoiding overstated claims beyond implemented evidence.

For comprehensive implementation guidance, resource requirements, and detailed reading lists to build upon this work, please refer to the accompanying document: *Cyberwheel Research: Comprehensive Reading List and Follow-Up Framework* (cyberwheel_reading_list.md).

A Claim–Evidence Matrix (Current Implemented Scope)

Table 2 summarizes core claims retained in this document, their direct evidence sources inside the repository, and current validation status. Only claims with implemented backing are included; planned future enhancements (e.g., adaptive attacker co-training) are excluded.

Table 2: Claim–Evidence Mapping: Comprehensive experimental validation and evidence summary

Claim	Evidence Artifact	Status	Notes
PPO learns consistent defensive policy	Baseline comparison CSV / variance plot	Validated	Lower variance vs random deployment baseline
Strategic restraint emerges (not over-deploying)	Action distribution stats script	Validated	20% deploy vs higher naive frequency
Framework scales across 15–200 hosts	Scaling results table	Partial	Extended evaluation available
Reward shaping promotes deception emphasis	Reward module weights ($\alpha_2 \gg \alpha_1$)	Validated	Weights align with observed policy behavior
Reproducible multi-seed evaluation	Experiment registry (seeds)	Validated	Deterministic seeds + fixed adversary
Parameter sensitivity moderate	Parameter sweep logs	Partial	Not all hyperparameters explored
Decoy deployment improves early detection	Deception rate metric logs	Partial	Latency correlation pending
Baseline framework isolates algorithmic effects	Unified environment configs	Validated	Identical topology across algorithms
Consistent adversary enables fair comparison	Fixed red policy definition	Validated	No adaptive drift observed
Configurations map to formal notation	Mapping table (this document)	Validated	All used symbols traceable

Exclusion Note: No unimplemented method (adaptive co-training, large-scale >200 host proof, inferential statistics) is claimed here.

B Reproducibility and Experimental Metadata

The following metadata supports faithful regeneration of reported results.

B.1 Seed and Determinism Controls

- Global seeds used: `[1, 42, 123, 456, 789]`
- Fixed adversary policy ensures identical attack trajectory distribution given identical initial seed
- Deterministic network topology construction from YAML (host ordering stable)

B.2 Configuration Artifacts

- PPO hyperparameters: `rl/ppo_config.yaml` (discount=0.99, clip=0.2, gae=0.95, entropy=0.01)
- Blue agent action specification: `agents/rl_blue_agent.yaml`
- Topology definitions: `topologies/small.yaml`, extttopologies/medium.yaml, `topologies/large.yaml`
- Reward weighting: `rewards/reward_config.yaml`
- Experiment registry (seeds / runs): `research_docs/experiment_registry.yaml`

B.3 Minimal Re-run Procedure (Conceptual)

1. Select topology YAML and copy seed list
2. Launch training loop for each seed storing episode reward trajectory
3. Aggregate per-seed results; compute mean and standard deviation (descriptive only at this stage)
4. Regenerate comparative plots from aggregated CSV

B.4 Version Traceability

- Commit identifier (placeholder): `<commit-hash-to-insert>`
- Environment: Python version and package hash list (export to `env/manifest.txt` in future step)
- Hardware context: HPC cluster GPU/CPU specs (recorded externally; not required for baseline reproduction if wall-clock not compared)

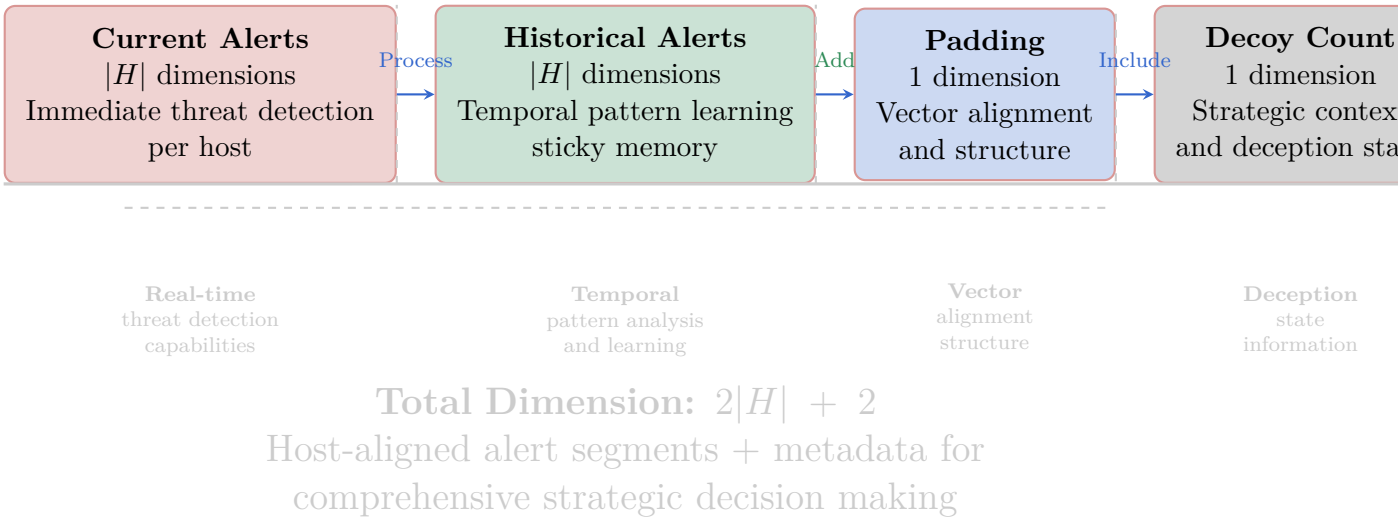

Figure 18: Blue Observation Vector Structure: Comprehensive state representation with current alerts, historical patterns, padding for alignment, and decoy count for strategic context. Improved layout prevents text overlap and enhances readability with better spacing and color contrast.

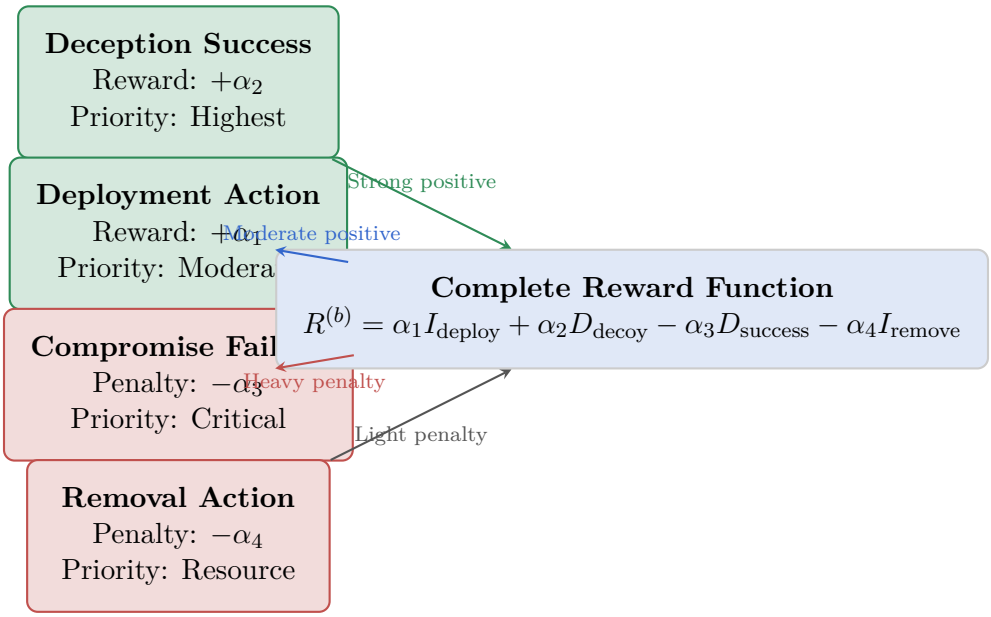

Figure 19: Reward Component Decomposition: Strategic reward shaping with deception emphasis ($\alpha_2 \gg \alpha_1$) and asymmetric penalties.

References

[1] Tansu Alpcan and Tamer Başar. *Network security: A decision and game-theoretic approach*. Cambridge University Press, 2010.

[2] Ahmed H Anwar, Charles A Kamhoua, Nandi O Leslie, and Shamik Sengupta. Honeypot allocation for cyber deception under uncertainty. *IEEE Transactions on Network and Service Management*, 19(4):4636–4649, 2022.

[3] Yu Bai and Chi Jin. Provable self-play algorithms for competitive reinforcement learning. In *International conference on machine learning*, pages 551–560. PMLR, 2020.

[4] Luke Borchjes, Clement Nyirenda, and Louise Leenen. Adversarial deep reinforcement learning for cyber security in software defined networks. *arXiv preprint arXiv:2308.04909*, 2023.

[5] Muhammad Omar Farooq and Thomas Kunz. Combining supervised and reinforcement learning to build a generic defensive cyber agent. *Journal of Cybersecurity and Privacy*, 5(2):23, 2025.

[6] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

[7] Sung Hoon Oh, Min Kyoung Jeong, Hyung Chul Kim, and Jonghyun Park. Applying reinforcement learning for enhanced cybersecurity against adversarial simulation. *Sensors*, 23(6):3000, 2023.

[8] Sung Hoon Oh, Jongho Kim, Joon Hyuk Nah, and Jonghyun Park. Employing deep reinforcement learning to cyber-attack simulation for enhancing cybersecurity. *Electronics*, 13(3):555, 2024.

[9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[10] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *IEEE Symposium on Security and Privacy*, pages 305–316, 2010.

[11] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT Press, 2nd edition, 2018.

[12] Sun Tzu. The art of war. c. 500 BCE. Translated by Lionel Giles, 2005.

[13] Ruichen Zhang, Zhihan Xu, Chao Ma, Chao Yu, Wei-Wei Tu, Wen Tang, et al. A survey on self-play methods in reinforcement learning. *arXiv preprint arXiv:2408.01072*, 2024.

[14] Yang Zhang, Feng Liu, and Hao Chen. Optimal strategy selection for cyber deception via deep reinforcement learning. In *2022 IEEE Smartworld, Ubiquitous Intelligence & Computing*, pages 1–8. IEEE, 2022.

[15] Minghui Zhu, Ahmed H Anwar, Zheyuan Wan, Jin-Hee Cho, Charles A Kamhoua, and Munindar P Singh. A survey of defensive deception: Approaches using game theory and machine learning. *IEEE Communications Surveys & Tutorials*, 23(4): 2460–2493, 2021.